مـدن الـمـهـن و الـكـفـاءات
Cités des métiers et des compétences

مكتب التكوين المهني و إنعاش الشغل
Office de la Formation Professionnelle et de la Promotion du Travail

Module: Développer en back-end

Base de données
Migration

Filière: Développement digital – option web full stack

Développer en back-end

C. Approfondir la programmation Laravel

2. Interagir avec la base de données

Gestion de Migration

Introduction

Une migration permet de créer et de mettre à jour un schéma de base de données. Autrement dit, vous pouvez créer des tables, des colonnes dans ces tables, en supprimer, créer des index… Tout ce qui concerne la maintenance de vos tables peut être pris en charge par cet outil. Vous avez ainsi un suivi de vos modifications.

La migration Laravel est comme un outil de gestion de version de DATABASE.

Développer en back-end

C. Approfondir la programmation Laravel

2. Interagir avec la base de données

Gestion de Migration

La configuration de la base

Laravel permet de gérer les bases de type **MySQL**, **Postgres**, **SQLite** et **SQL Server**.

Il faut indiquer où se trouve votre base, son nom, le nom de l'utilisateur, le mot de passe dans le fichier de configuration **.env** :

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=laravel
DB_USERNAME=root
DB_PASSWORD=
```

Ici nous avons les valeurs par défaut à l'installation de Laravel. Il faudra évidemment modifier ces valeurs selon votre contexte de développement et définir surtout le nom de la base, le nom de l'utilisateur et le mot de passe. Pour une installation de **MySQL** en local en général l'utilisateur est **root** et on n'a pas de mot de passe.

Développer en back-end

C. Approfondir la programmation Laravel

2. Interagir avec la base de données

Gestion de Migration

Installation

Si vous regardez dans le dossier database/migrations il y a déjà 4 migrations présentes :

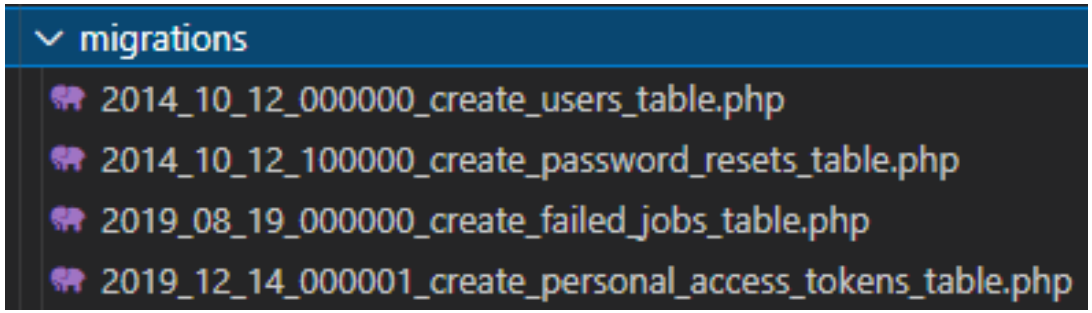


table users : c'est une migration de base pour créer une table des utilisateurs,

table password_resets : c'est une migration liée à la précédente qui permet de gérer le renouvellement des mots de passe en toute sécurité,
table failed_jobs : une migration qui concerne les queues,

table personal_access_tokens concerne les api.

Développer en back-end

C. Approfondir la programmation Laravel

2. Interagir avec la base de données

Gestion de Migration

Exécution de migrations

- ✓ lancer les migrations on utilise la commande **migrate** :

```
php artisan migrate
```

- ✓ Annuler la dernière migration, nous avons la commande

```
php artisan migrate:rollback
```

- ✓ Annuler la migration jusqu'à **n** étapes en commençant par la plus récente.

```
php artisan migrate:rollback --step=n
```

- ✓ Annuler toutes les opérations on utilise la commande

```
php artisan migrate:reset
```

- ✓ Afficher l'état des migrations on utilise la commande

```
php artisan migrate:status
```

❖ *Les méthodes **down** des migrations sont exécutées et les tables sont supprimées*.

Développer en back-end

C. Approfondir la programmation Laravel

2. Interagir avec la base de données

Gestion de Migration

Exécution de migrations

✓ Annuler toutes vos migrations, puis exécutera la commande migrate.

Cette commande recrée efficacement l'intégralité de votre base de données :

```
php artisan migrate:refresh
```

✓ Annuler et migrer à nouveau un nombre limité de migrations.

```
php artisan migrate:refresh –step=n
```

✓ Pour éviter d'avoir à coder la méthode **down** on a la commande **fresh** qui supprime automatiquement les tables concernées :

Supprimera toutes les tables de la base de données, puis exécutera la commande migrate :

```
php artisan migrate:fresh
```

NB : refresh=(rollback all + migrate) *fresh=(drop all + migrate)*

C. Approfondir la programmation Laravel

2. Interagir avec la base de données

Gestion de Migration

Créer une migration

Pour générer une migration, vous devez exécuter une commande

```
php artisan make:migration create_stagiaires_table
```

Cela générera un fichier dans le dossier **database\migrations**

Le fichier consiste en une nouvelle classe étendant la classe de migration de LARAVEL.

On dispose dans cette classe de deux fonctions :

up : ici on a le code de création de la table et de ses colonnes

down : ici on a le code de suppression de la table

C. Approfondir la programmation Laravel

2. Interagir avec la base de données

Gestion de Migration

Les fonctions (**up et down**)

```php
public function up()
    {
        Schema::create('stagiaires', function (Blueprint $table) {
            $table->id();
            $table->string('nom');
            $table->string('prenom', 50);
            //$table->boolean('etat')->default(0);
            $table->timestamps();
        });
    }
```

```php
public function down()
    {
        Schema::dropIfExists('stagiaires');
    }
```

Développer en back-end

C. Approfondir la programmation Laravel

2. Interagir avec la base de données

Gestion de Migration

Ajouter une colonne à une table

```
php artisan make:migration add_age_to_stagiaires
```

```
public function up()
 {
    Schema::table('stagiaires', function (Blueprint $table) {
            $table->integer('age')->after(('prenom'));
        });
    }
```

```
 public function down()
    {
        Schema::table('stagiaires', function (Blueprint $table) {
            $table->dropColumn('age');
        });
    }
```

Développer en back-end

C. Approfondir la programmation Laravel

2. Interagir avec la base de données

Gestion de Migration

Créer des clés étrangères à l'aide de migrations

Model logique de Donnée :



Développer en back-end

C. Approfondir la programmation Laravel

2. Interagir avec la base de données

Gestion de Migration

Créer des clés étrangères à l'aide de migrations

La table categories

```
public function up()
    {
        Schema::create('categories', function (Blueprint $table) {
        $table->id();
        $table->string('name')->unique();
        $table->string('slug')->unique();
        $table->timestamps();
        });
     }
```

C. Approfondir la programmation Laravel

2. Interagir avec la base de données

Gestion de Migration

Créer des clés étrangères à l'aide de migrations

La table films (1 ère façon)

```
public function up()
{
  Schema::disableForeignKeyConstraints();
  Schema::create('films', function (Blueprint $table) {
        $table->id();
        $table->foreign('category_id')->references('id')
                                      ->on('categories')
                                      //->cascadeOnDelete()
                                      //->cascadeOnUpdate()
                                      ->onDelete('restrict')
                                      ->onUpdate('restrict');

    });
}
```

C. Approfondir la programmation Laravel

2. Interagir avec la base de données

Gestion de Migration

Créer des clés étrangères à l'aide de migrations

La table films (2 -ème façon) *a partir de laravel 8 et plus*

```
public function up()
{
    Schema::disableForeignKeyConstraints();
    Schema::create('films', function (Blueprint $table) {
        ...
        $table->foreignId('category_id')->constrained()
                                        //->cascadeOnDelete()
                                        //->cascadeOnUpdate()
                                       ->onUpdate('restrict')
                                       ->onDelete('restrict');
});
}
```

Développer en back-end

C. Approfondir la programmation Laravel

2. Interagir avec la base de données

Gestion de Migration

Créer des clés étrangères à l'aide de migrations

Dans la table films on déclare une clé étrangère nommée **category_id** qui référence la colonne **id** dans la table **categories**.

La méthode **foreignId** crée une colonne de type **UNSIGNED BIGINT**.
La méthode **constrained** utilise les conventions de Laravel pour déterminer le nom de la table référencée, ici c'est **categories**.

En cas de suppression (**onDelete**) ou de modification (**onUpdate**) on a une restriction (**restrict**).

Une autre possibilité est **cascade** à la place de **restrict**.

Dans ce cas si vous supprimez une catégorie ça supprimera en cascade les films de cette catégorie.

C. Approfondir la programmation Laravel

2. Interagir avec la base de données

Gestion de Migration

Créer des clés étrangères à l'aide de migrations

Les migrations sont effectuées dans l'ordre alphabétique, ce qui peut générer un problème avec les clés étrangères. Si la table référencée est créée après.

C'est pour cette raison que j'ai ajouté cette ligne dans la migration :

```
Schema::disableForeignKeyConstraints();
```

On désactive temporairement le contrôle référentiel le temps de créer les tables.